

C++ 面向对象代码题汇总 (id:119-122)

适合打印复习：题目要点 + 完整代码 + 易错点

目录

- A. 机器人变身（类与对象）
- B. 虚拟电话（构造与析构）
- C. 银行账户（拷贝构造）
- D. 电视遥控（静态成员 + 友元函数）

说明：以下代码均按对应样例输入输出核对过；D 题音量处理按样例采用 0 到 100 截断。

A. 机器人变身（类与对象）

题目核心：根据机器人类型和等级计算血量、伤害、防御；全局变身函数接收对象指针，类型不同才变身并计数。

关键点：

- 普通型 N：血量、伤害、防御都是 level * 5。
- 攻击型 A：只有伤害变为 level * 10。
- 防御型 D：只有防御变为 level * 10。
- 生命型 H：只有血量变为 level * 50。
- 把属性刷新集中写在 updateAttr()，避免构造和变身时重复计算。

完整代码：

```
#include <iostream>
#include <string>
using namespace std;

class Robot {
private:
    string name;
    char type;
    int level;
    int hp;
    int attack;
    int defense;

    void updateAttr() {
        hp = level * 5;
        attack = level * 5;
        defense = level * 5;

        if (type == 'A') {
            attack = level * 10;
        } else if (type == 'D') {
            defense = level * 10;
        } else if (type == 'H') {
            hp = level * 50;
        }
    }

public:
    Robot(string n, char t, int lv) {
        name = n;
        type = t;
        level = lv;
        updateAttr();
    }

    string getName() {
        return name;
    }

    char getType() {
        return type;
    }

    int getLevel() {
        return level;
    }

    int getHp() {
        return hp;
    }

    int getAttack() {
        return attack;
    }

    int getDefense() {
        return defense;
    }

    void setType(char t) {
        type = t;
        updateAttr();
    }
};
```

```

    }

    void show() {
        cout << name << "--" << type << "--" << level << "--"
            << hp << "--" << attack << "--" << defense << endl;
    }
};

bool transform(Robot *r, char newType) {
    if (r->getType() == newType) {
        return false;
    }

    r->setType(newType);
    return true;
}

int main() {
    int t;
    cin >> t;

    int transformCount = 0;

    while (t--) {
        string name;
        char type, newType;
        int level;

        cin >> name >> type >> level;
        cin >> newType;

        Robot robot(name, type, level);

        if (transform(&robot, newType)) {
            transformCount++;
        }

        robot.show();
    }

    cout << "The number of robot transform is " << transformCount << endl;

    return 0;
}

```

B. 虚拟电话（构造与析构）

题目核心：Phone 类中包含 PhoneNumber 类对象，构造时输出 constructed，析构时按对象销毁顺序输出 destructed。

关键点：

- PhoneNumber 是复合类成员，建议用初始化列表构造。
- 三个对象 p1、p2、p3 的析构顺序与构造顺序相反：p3、p2、p1。
- 查询时依次判断三个对象，找到就 print()，找不到输出 wrong number。
- 状态 1 输出 use，状态 0 输出 unuse。

完整代码：

```
#include <iostream>
#include <string>
using namespace std;

class PhoneNumber {
private:
    int number;
    char type;
public:
    PhoneNumber(int n = 0, char t = 'C') {
        number = n;
        type = t;
    }

    int getNumber() {
        return number;
    }

    char getType() {
        return type;
    }

    void setNumber(int n) {
        number = n;
    }

    void setType(char t) {
        type = t;
    }
};

class Phone {
private:
    PhoneNumber phoneNumber;
    int state;
    string owner;
public:
    Phone(int num, char type, int st, string name)
        : phoneNumber(num, type) {
        state = st;
        owner = name;

        cout << phoneNumber.getNumber() << " constructed." << endl;
    }

    ~Phone() {
        cout << phoneNumber.getNumber() << " destructed." << endl;
    }

    int query(int num) {
        if (phoneNumber.getNumber() == num) {
            return 1;
        }
        return 0;
    }

    void print() {
        cout << "Phone=" << phoneNumber.getNumber()
            << "--Type=" << phoneNumber.getType()
            << "--State=";
```

```

        if (state == 1) {
            cout << "use";
        } else {
            cout << "unuse";
        }

        cout << "--Owner=" << owner << endl;
    }
};

int main() {
    int num1, num2, num3;
    char type1, type2, type3;
    int state1, state2, state3;
    string owner1, owner2, owner3;

    cin >> num1 >> type1 >> state1 >> owner1;
    cin >> num2 >> type2 >> state2 >> owner2;
    cin >> num3 >> type3 >> state3 >> owner3;

    Phone p1(num1, type1, state1, owner1);
    Phone p2(num2, type2, state2, owner2);
    Phone p3(num3, type3, state3, owner3);

    int t;
    cin >> t;

    while (t--) {
        int q;
        cin >> q;

        if (p1.query(q)) {
            p1.print();
        } else if (p2.query(q)) {
            p2.print();
        } else if (p3.query(q)) {
            p3.print();
        } else {
            cout << "wrong number." << endl;
        }
    }

    return 0;
}

```

C. 银行账户（拷贝构造）

题目核心：先创建活期账户，再通过拷贝构造创建定期账户；定期账户账号加 50000000，利率改为 0.015。

关键点：

- 活期账户利率为 0.005，定期账户利率为 0.015。
- Account fixed(current); 会调用拷贝构造函数。
- C 操作只输出计息后的账号和余额。
- P 操作输出账号、类型、余额、利率。

完整代码：

```
#include <iostream>
#include <string>
using namespace std;

class Account {
private:
    int account;
    char type;
    double sum;
    double rate;

public:
    Account(int acc, char t, double s) {
        account = acc;
        type = t;
        sum = s;
        rate = 0.005;
    }

    Account(const Account &a) {
        account = a.account + 50000000;
        type = a.type;
        sum = a.sum;
        rate = 0.015;
    }

    void calculateInterest() {
        sum = sum + sum * rate;
        cout << "Account=" << account << "--sum=" << sum << endl;
    }

    void print() {
        cout << "Account=" << account << "--";

        if (type == 'P') {
            cout << "Person";
        } else if (type == 'E') {
            cout << "Enterprise";
        }

        cout << "--sum=" << sum << "--rate=" << rate << endl;
    }
};

int main() {
    int t;
    cin >> t;

    while (t--) {
        int account;
        char type;
        double sum;

        cin >> account >> type >> sum;

        Account current(account, type, sum);
        Account fixed(current);

        char op1, op2;
        cin >> op1 >> op2;

        if (op1 == 'C') {
            current.calculateInterest();
        } else if (op1 == 'P') {
            current.print();
        }
    }
}
```

```
    }  
    if (op2 == 'C') {  
        fixed.calculateInterest();  
    } else if (op2 == 'P') {  
        fixed.print();  
    }  
}  
return 0;  
}
```

D. 电视遥控（静态成员 + 友元函数）

题目核心：用动态数组创建 n 台电视；静态成员统计当前 TV/DVD 数量；友元遥控函数能直接修改私有成员。

关键点：

- 创建完 n 台电视后，初始状态是 TV 数量 n，DVD 数量 0。
- 只有模式真正变化时才更新 tvCount 和 dvdCount。
- DVD 模式频道固定为 99；TV 模式频道使用输入频道。
- 题面说越界不变，但样例显示应截断：音量大于 100 置 100，小于 0 置 0。
- 静态成员变量必须在类外初始化。

完整代码：

```
#include <iostream>
using namespace std;

class Television {
private:
    int id;
    int volume;
    int mode;
    int channel;

    static int tvCount;
    static int dvdCount;

public:
    Television() {
        id = 0;
        volume = 50;
        mode = 1;
        channel = 0;
    }

    void init(int i) {
        id = i;
        volume = 50;
        mode = 1;
        channel = i;
        tvCount++;
    }

    void print() {
        cout << "第" << id << "号电视机--";

        if (mode == 1) {
            cout << "TV 模式";
        } else {
            cout << "DVD 模式";
        }

        cout << "--频道" << channel
            << "--音量" << volume << endl;
    }

    static void printCount() {
        cout << "播放的机量" << tvCount << endl;
        cout << "播放 DVD 的电视机数量为" << dvdCount << endl;
    }

    friend void remoteControl(Television &tv, int newMode, int newChannel, int changeVolume);
};

int Television::tvCount = 0;
int Television::dvdCount = 0;

void remoteControl(Television &tv, int newMode, int newChannel, int changeVolume) {
    if (tv.mode != newMode) {
        if (tv.mode == 1) {
            Television::tvCount--;
        } else {
            Television::dvdCount--;
        }

        if (newMode == 1) {
            Television::tvCount++;
        }
    }
}
```



```

        } else {
            Television::dvdCount++;
        }
    }

    tv.mode = newMode;

    if (newMode == 1) {
        tv.channel = newChannel;
    } else {
        tv.channel = 99;
    }

    int newVolume = tv.volume + changeVolume;

    if (newVolume > 100) {
        tv.volume = 100;
    } else if (newVolume < 0) {
        tv.volume = 0;
    } else {
        tv.volume = newVolume;
    }

    tv.print();
}

int main() {
    int n;
    cin >> n;

    Television *tv = new Television[n + 1];

    for (int i = 1; i <= n; i++) {
        tv[i].init(i);
    }

    int t;
    cin >> t;

    while (t--) {
        int i, k, x, v;
        cin >> i >> k >> x >> v;

        remoteControl(tv[i], k, x, v);
    }

    Television::printCount();

    delete[] tv;

    return 0;
}

```

统一复习提醒

- 私有成员不能在类外直接访问，需要通过成员函数或友元函数操作。
- 构造函数负责初始化对象；析构函数在对象生命周期结束时自动调用，不要手动调用。
- 拷贝构造函数常用于“根据已有对象生成新对象，但部分属性要调整”的场景。
- 静态成员变量属于类，不属于某一个对象；类外初始化格式：int ClassName::var = 0;
- 动态数组用 new[] 创建，就必须用 delete[] 释放。